

# **Modern Storages and Data Warehousing Week 4 - File storages**

Попов Илья, [i.popov@hse.ru](mailto:i.popov@hse.ru)

**1 - S3**

**Simple Storage Service**

# Что такое S3

- › S3 — объектное хранилище AWS, запущено в марте 2006 года
- › Проект стал настолько популярен, что де-факто стал стандартом в индустрии
- › Сторонние проекты начали делать свои хранилища с API, полностью совместимым с S3, чтобы облегчить жизнь разработчикам
- › Сейчас многие открытые и вендорские решения (MinIO, Ceph RGW, вендорские Object Storage) поддерживают S3-совместимый API
- › Так появились S3-like хранилища как класс

# Почему S3

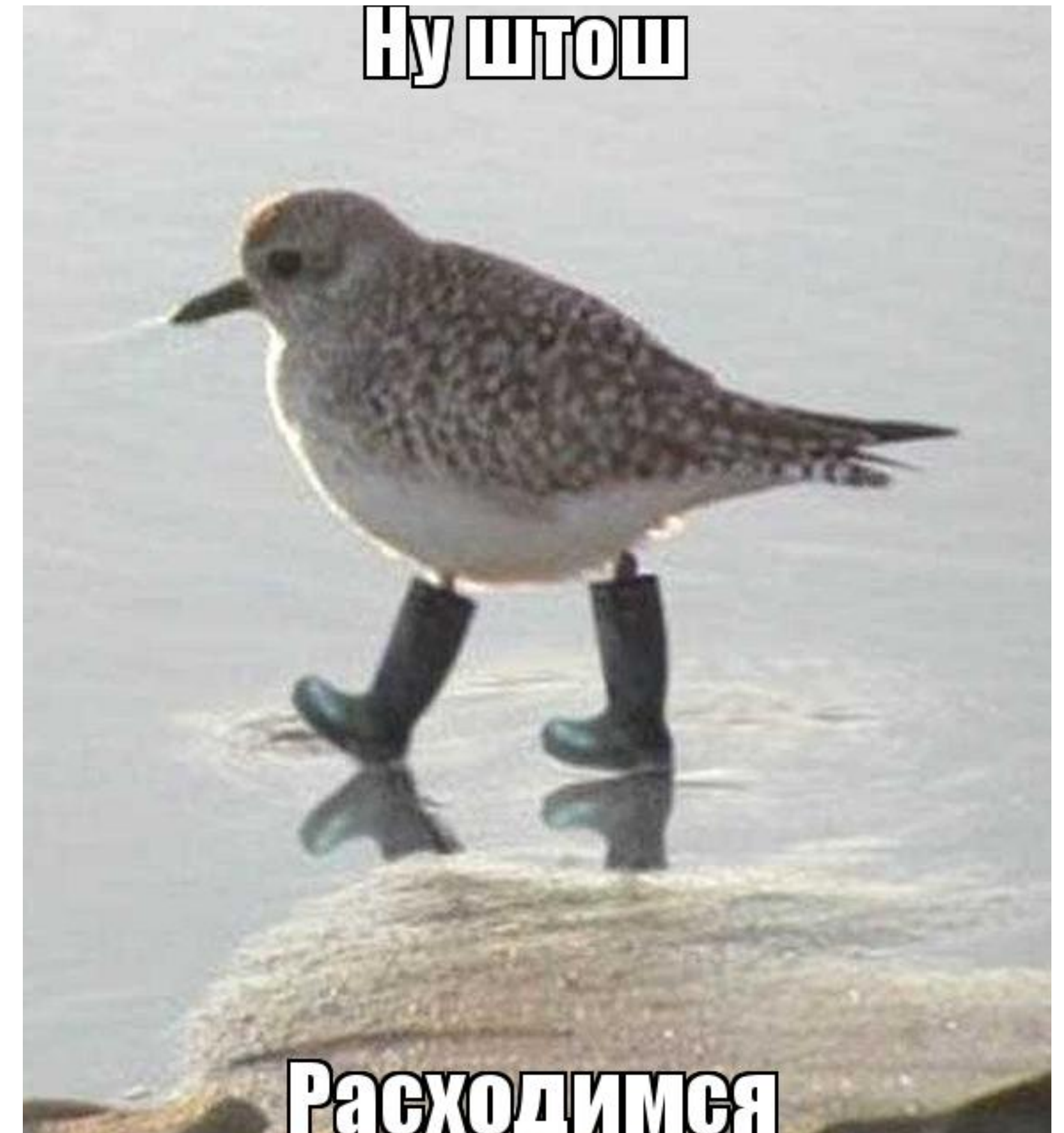
› Говорю S3 - подразумеваю любые **Object Storage** в принципе вне зависимости от реализации и вендора (AWS S3, Yandex Object Storage, MinIO, Ceph и так далее и тому подобное)

# Почему S3

- › Говорю S3 - подразумеваю любые **Object Storage** в принципе вне зависимости от реализации и вендора (AWS S3, Yandex Object Storage, MinIO, Ceph и так далее и тому подобное)
- › Дешевые и бездонные

# Почему S3

- › Говорю S3 - подразумеваю любые **Object Storage** в принципе вне зависимости от реализации и вендора (AWS S3, Yandex Object Storage, MinIO, Сeph и так далее и тому подобное)
- › Дешевые и бездонные



# Почему S3

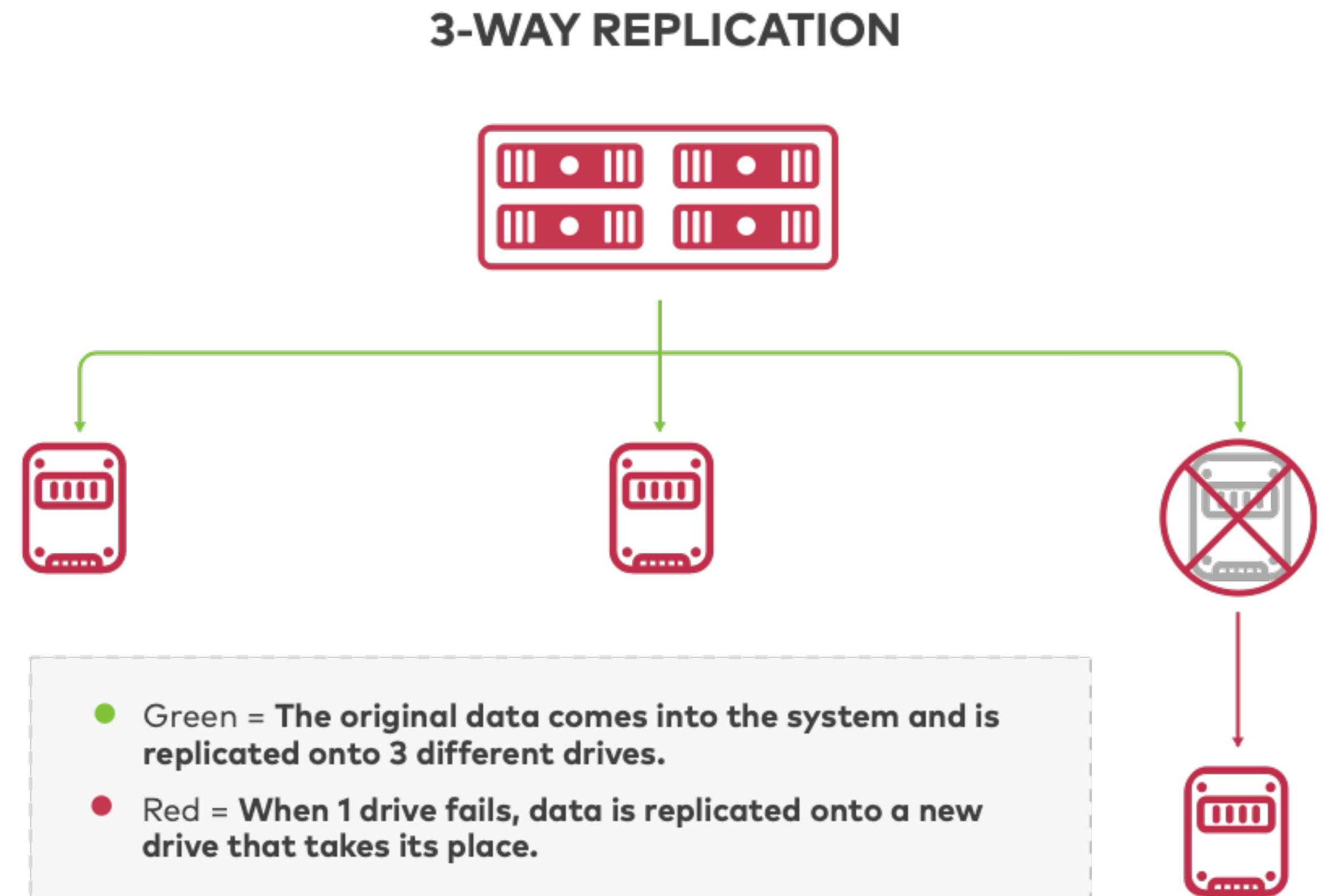
- › Дешевые и бездонные
- › SDK к S3 есть практически на любом языке программирования
- › Кроме классической отказоустойчивости через репликацию блоков, многие реализации также используют Erasure Coding
- › Могут раздавать статику напрямую по HTTP или служить источником для CDN

## Минусы:

- › Объекты неизменяемы: изменить объект = перезаписать его целиком
- › Умеют только сжимать, кодировать и/или шифровать данные, вычислять на них ничего не получится
- › AWS стоит денег (и русские деньги больше не принимает)
- › **Но как правило Object Storage - самое дешевое КХД**

# Непонятное слово - Erasure Coding

- › Классический способ защиты от потери данных в распределённых хранилищах - репликация
- › Хранятся несколько полных копий данных на разных дисках/узлах (обычно на трех)
- › Если один выходит из строя - его меняют, после чего массив восстанавливается без даунтайма системы



# Непонятное слово - Erasure Coding

- › Данные бьются на более мелкие блоки (биты, байты или блоки) - data blocks
- › Блоки данных раскидываются по разным дискам (файлам)
- › Для нескольких ячеек данных насчитывается и сохраняется некоторое количество ячеек четности - parity blocks
- › Ошибка в любой чередующейся ячейке устраняется через обратную операцию декодирования на основе сохранившихся данных и ячеек четности
- › 3× репликация для 6 блоков - 18 блоков (перерасход 200%)  
EC RS(6,3) - 9 блоков (перерасход 50%).  
Обе схемы переживают потерю нескольких блоков: репликация - 2, RS(6,3) - 3
- › Чаще всего для вычисления блоков четности используется алгоритм Рида-Соломона
- › Потребляет CPU и внутрикластерную сеть

# Как хранятся данные в S3

- › Данные хранятся в **бакетах** - это контейнер верхнего уровня.
- › Внутри него пространство имён плоское: физически папок нет, только префиксы в ключе объекта
- › К бакету можно применять **политики** - правила доступа к данным
- › В бакет складываются **объекты** - произвольные файлы
- › Файлы можно дополнительно **каталогизировать**: добавлять теги, метаданные, настраивать время хранения и правила жизненного цикла

Всё \\_(ツ)\_/

Нам S3 нужен для:

- › Сохранить данные от пользователя
- › Раздать контент на фронт сервиса
- › Сохранить что-то далеко, дешево и надолго

# Как хранятся данные в S3

- › Данные хранятся в бакетах - это контейнер верхнего уровня.
- › Внутри него пространство имён плоское: физически папок нет, только префиксы в ключе объекта
- › К бакету можно применять **политики** - правила доступа к данным
- › В бакет складываются **объекты** - произвольные файлы
- › Файлы можно дополнительно **каталогизировать**: добавлять теги, метаданные, настраивать время хранения и правила жизненного цикла

Всё \\_(ツ)\_/

Есть еще **межкластерная** и **межрегионная репликация**, но те из вас кто не будут строить High Load CDN для сервиса уровня YouTube, Instagram или Amazon, скорее всего никогда с этим не столкнутся.

**Демо 1**

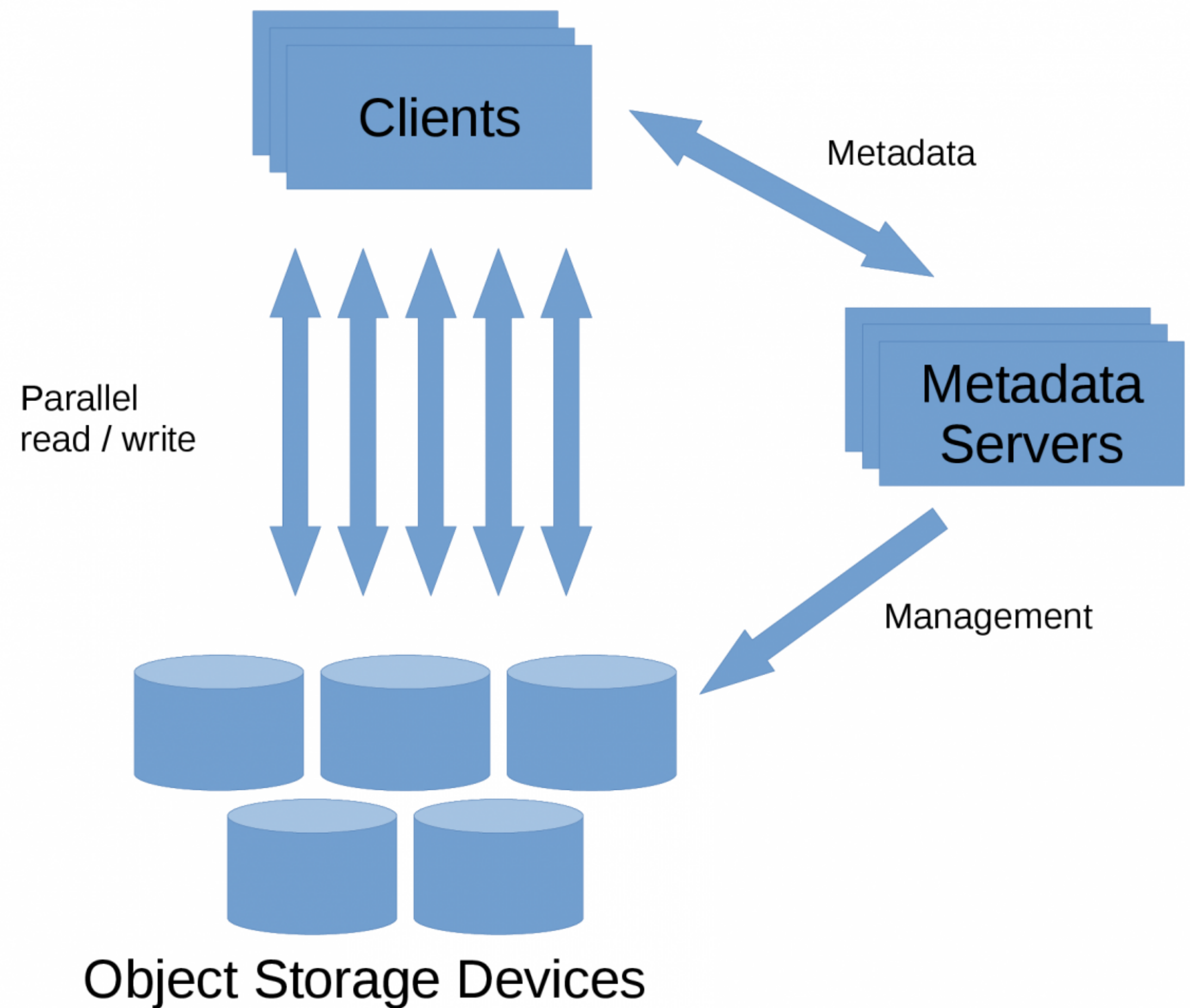
# **2 - Hadoop экосистема**

# Мотивация

- › Помним, что мы не хотим, чтобы аналитики ходили в OLTP
- › Помним, что мы не хотим хранить в OLTP избыточный объем истории
- › Помним, что одного хоста нам не хватает для эффективной обработки больших данных, и мы хотим распределять вычисления

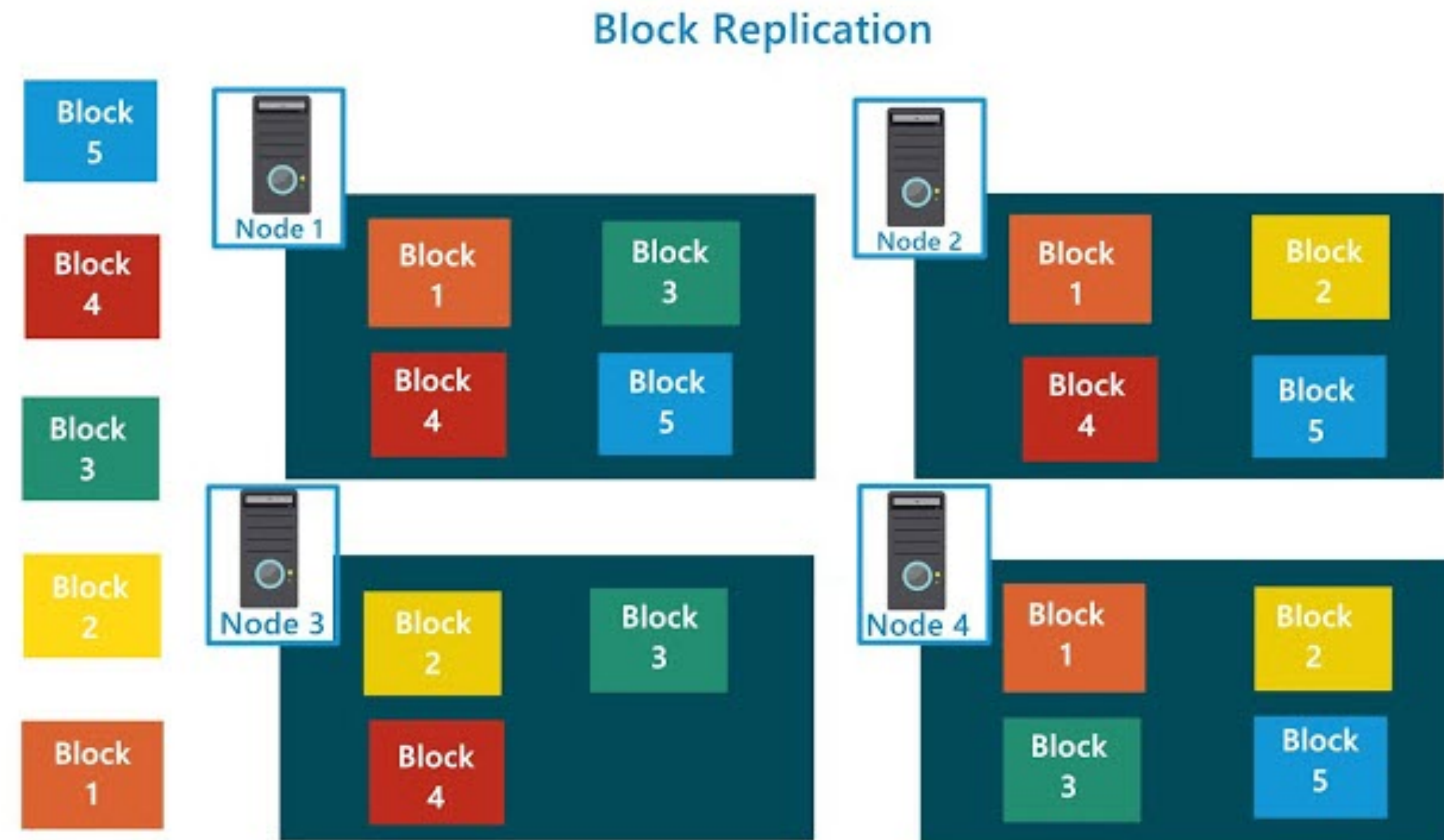
# Берем то, что уже видели в S3 и немного меняем

- › На одну машину все не поместится
- › Хотим иметь возможность не только чтения и полной перезаписи, но и уметь дописывать в конец файла (append)
- › Хотим распределенное хранилище без bottle neck в виде подключения к одной ноде



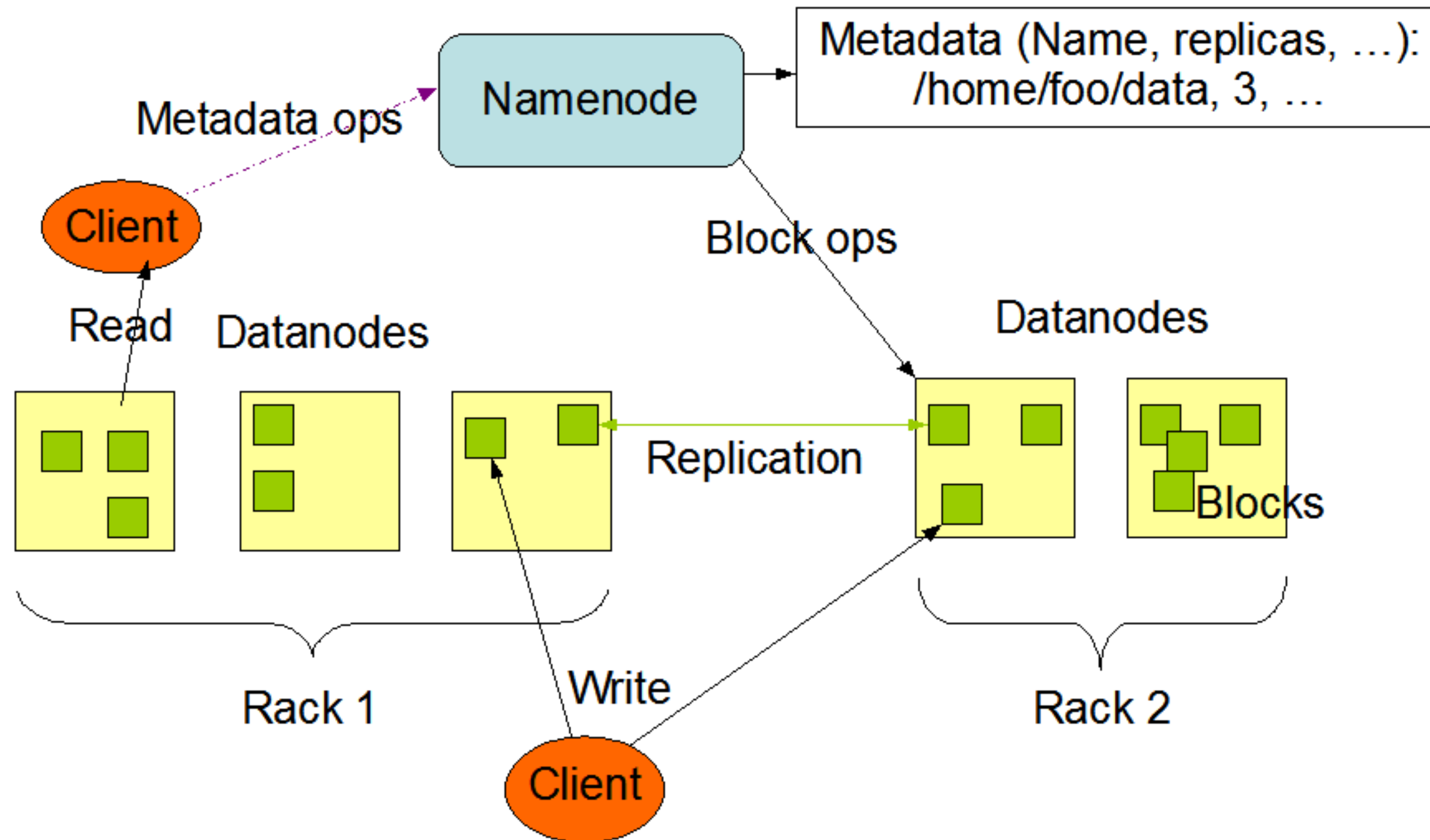
# Distributed File Storage

- › Хотим, чтобы хранилище было устойчивое и не теряло данные
- › Любая машина может сломаться в любой момент
- › Дробим данные на блоки, а блоки разносим по разным машинам так, чтобы при выходе одной из машин данные оставались целостными

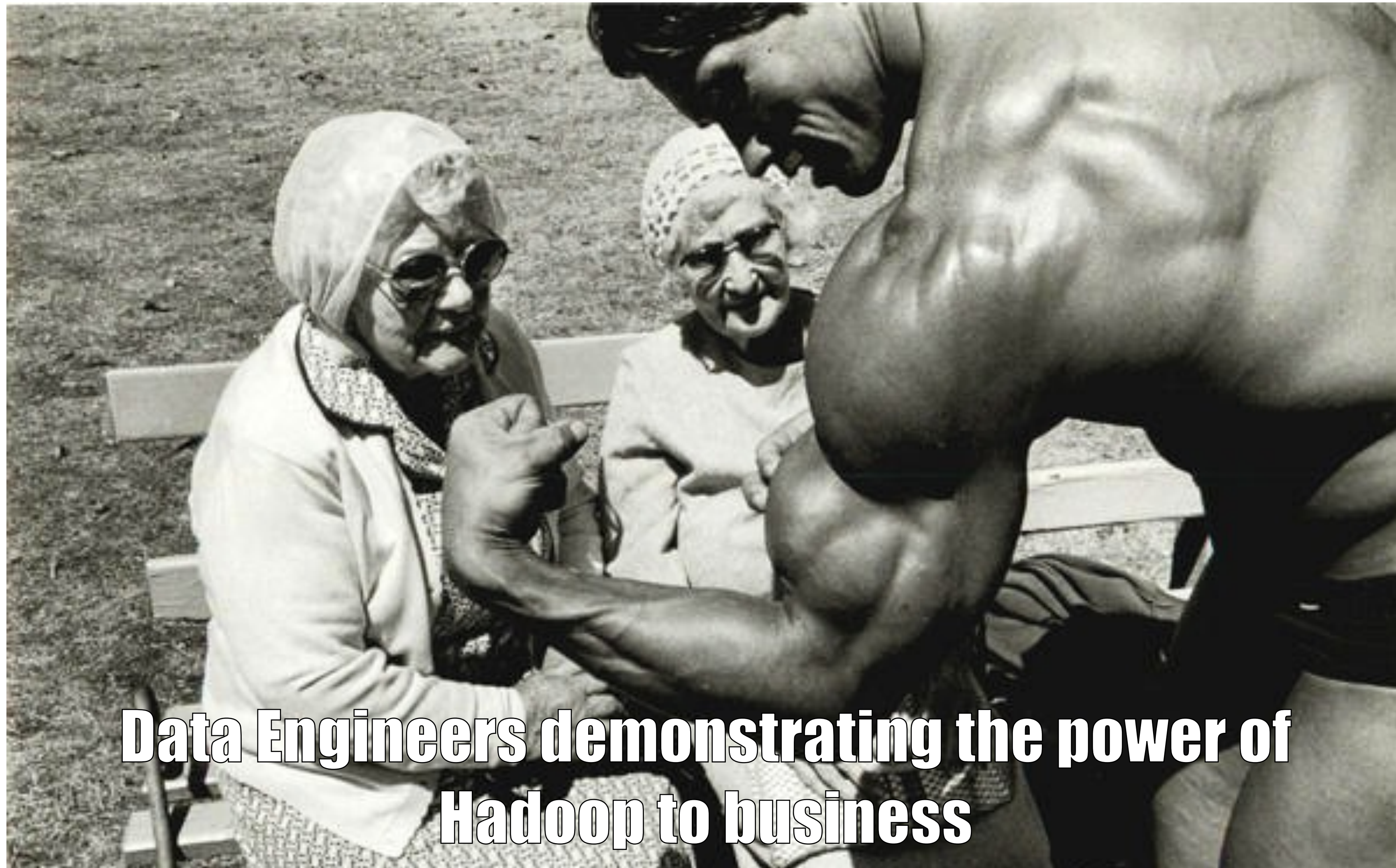


# Please welcome - HDFS

HDFS Architecture

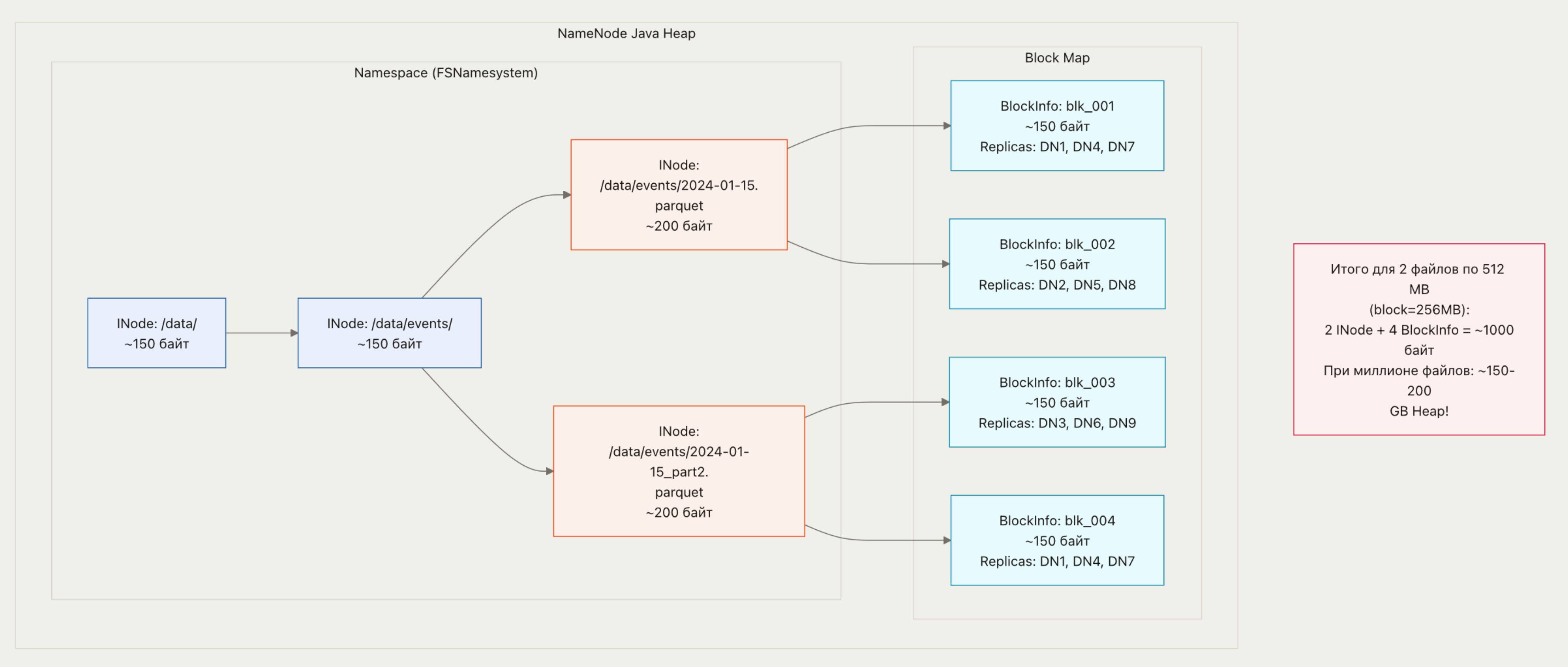


# Please welcome - HDFS



**Data Engineers demonstrating the power of  
Hadoop to business**

# Как хранятся данные в NameNode



# Проблема мелких файлов

Сценарий А: 1000 файлов по 1 GB (размер блока 256 MB)

Файлов: 1000

Блоков на файл:  $1\text{ GB} / 256\text{ MB} = 4$

Итого блоков:  $1000 \times 4 = 4000$

Heap NameNode:

- INode для 1000 файлов:  $1000 \times 200 = 200\text{ KB}$

- BlockInfo для 4000 блоков:  $4000 \times 150 = 600\text{ KB}$

Итого: ~800 KB

› Рассмотрим два сценария с одинаковым объёмом данных - 1 TB:

› Сценарий А: 1000 файлов по 1 GB (размер блока 256 MB)

# Проблема мелких файлов

Сценарий А: 1000 файлов по 1 GB (размер блока 256 MB)

Файлов: 1000  
Блоков на файл:  $1\text{ GB} / 256\text{ MB} = 4$   
Итого блоков:  $1000 \times 4 = 4000$

Heap NameNode:

- INode для 1000 файлов:  $1000 \times 200 = 200\text{ KB}$
- BlockInfo для 4000 блоков:  $4000 \times 150 = 600\text{ KB}$

Итого: ~800 KB

Сценарий В: 10 000 000 файлов по 100 KB (мелкие файлы)

Файлов: 10 000 000  
Блоков на файл: 1 (100 KB < 128 MB - один неполный блок)  
Итого блоков: 10 000 000

Heap NameNode:

- INode для 10M файлов:  $10\text{M} \times 200 = 2\text{ GB}$
- BlockInfo для 10M блоков:  $10\text{M} \times 150 = 1.5\text{ GB}$

Итого: ~3.5 GB

При репликации 3x (DatanodeStorageInfo  $\times$  3):

- Дополнительно:  $10\text{M} \times 3 \times 50 = 1.5\text{ GB}$

ИТОГО: ~5 GB только на метаданные

- › Рассмотрим два сценария с одинаковым объёмом данных - 1 TB:
- › Сценарий А: 1000 файлов по 1 GB (размер блока 256 MB)
- › Сценарий В: 10 000 000 файлов по 100 KB (мелкие файлы)
- › Оба сценария хранят одинаковый объём данных (1 TB), но потребление Heap NameNode отличается в 6000 раз.
- › При реальных масштабах (100 миллионов мелких файлов) NameNode требует 50–100 GB Heap только под метаданные.

**Демо 2.1**

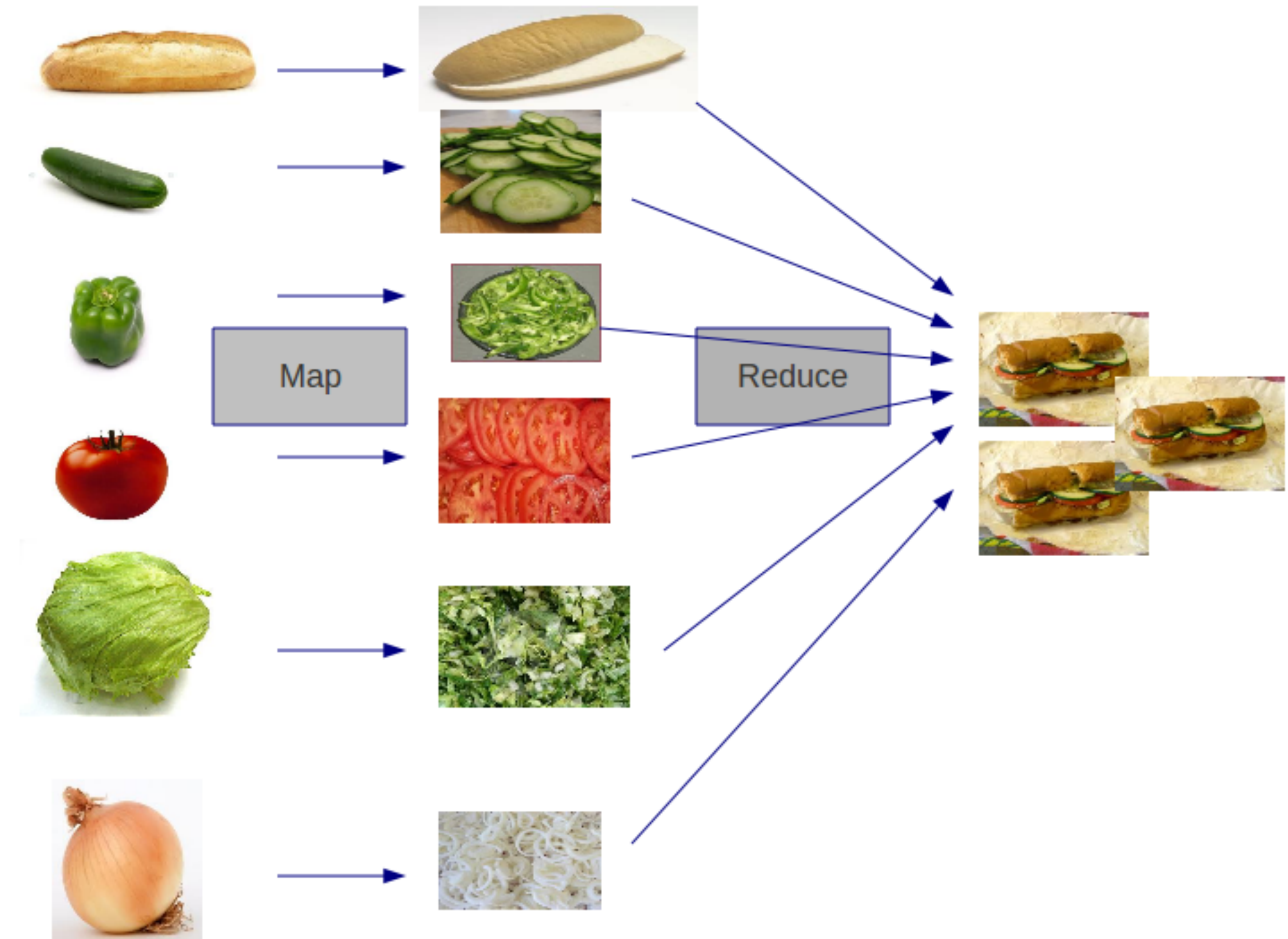
**Правило анализа данных #1:**  
**Если что-то можно посчитать**  
**на базе, а не на нашей**  
**машине - считаем на базе**

# Compute Engine

- › Хотим запускать вычисления над распределенными данными
- › Хотим чтобы эти вычисления тоже были распределенные
- › Нам нужна какая-то платформа, которая будет координировать выполнение кода
- › Нам нужна какая-то платформа, которая будет что-то делать, если одна из машин в кластере умерла
- › Нам нужна какая-то платформа, которая будет решать, как пересылать данные между машинами для агрегации
- › Хотим написать для всего этого фреймворк один раз и больше никогда об этом не думать

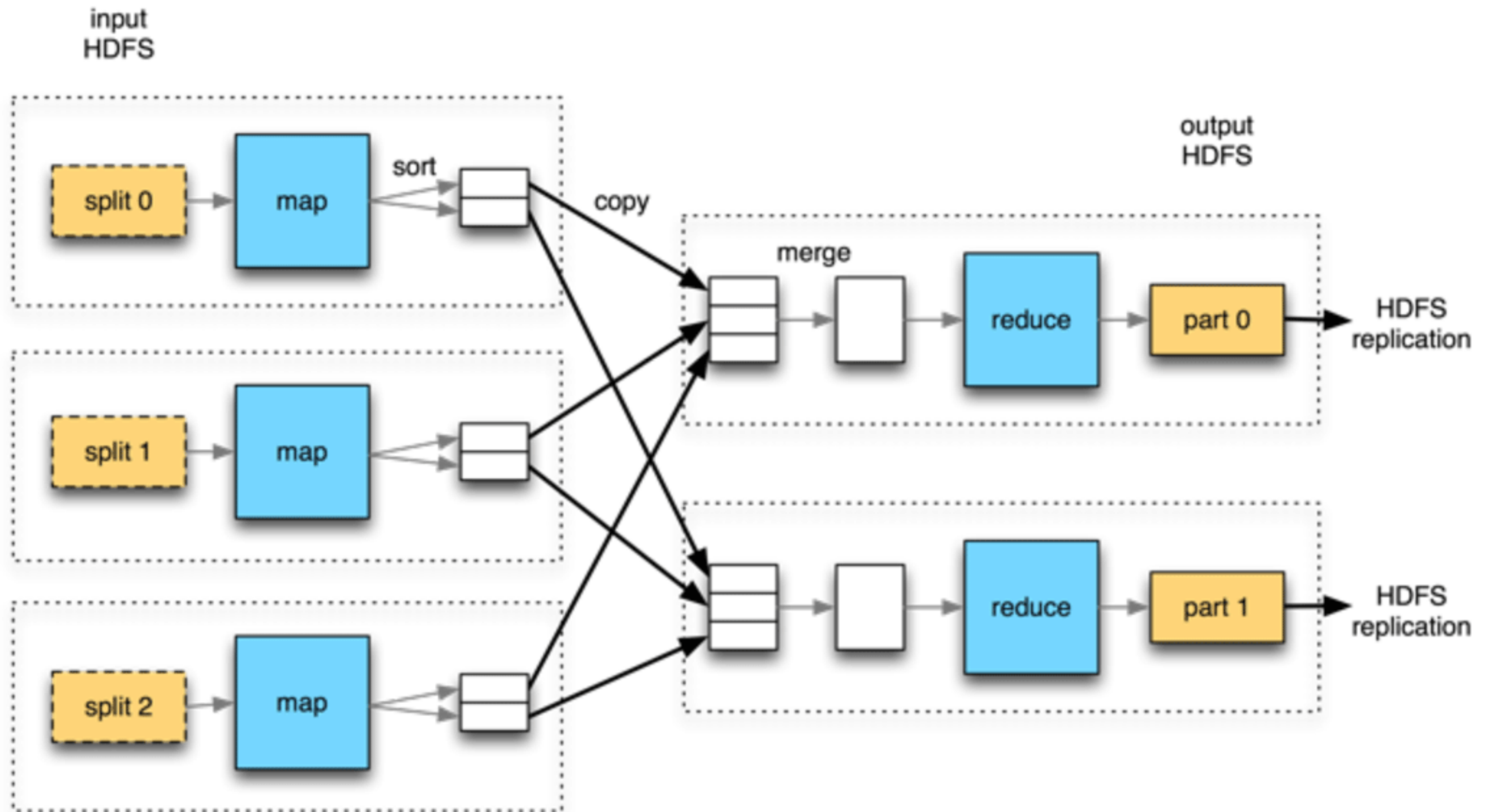
# Please welcome - Map Reduce

- › Бьем вычисления на фазы: Map, Shuffle/Sort, Reduce
- › Кластер за нас думает, где и как он будет запускать эти операции (помните, что у нас данных по 3 копии на разных нодах) и как делать shuffle
- › Мы радостно садимся думать, как же писать эти Map и Reduce



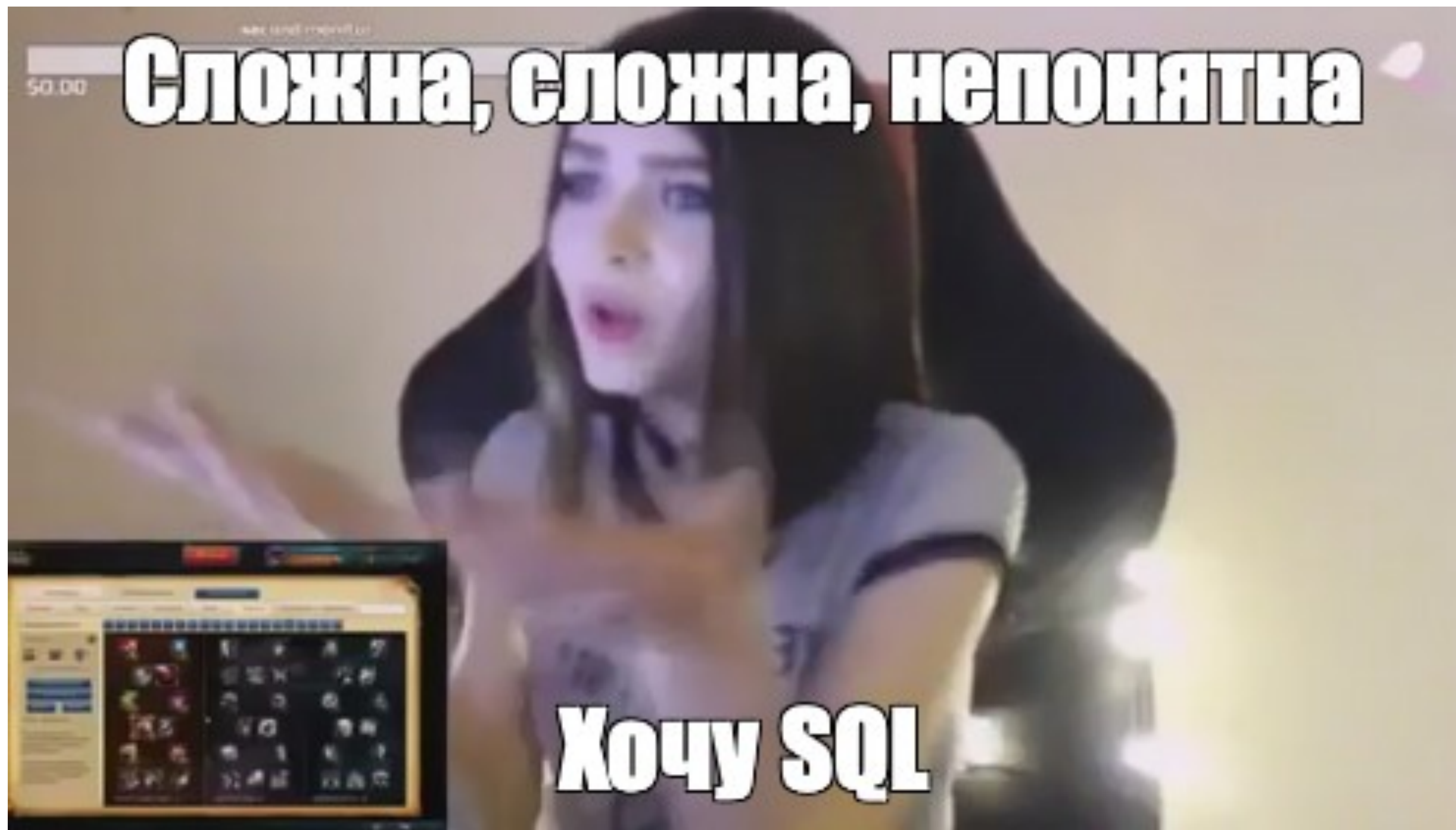
**Хрестоматийный пример -  
word count**

# Еще раз Map Reduce



# Какие тут есть проблемы?

# Какие тут есть проблемы?



# Please welcome - Spark

- › Spark - фреймворк для обработки больших данных.
- › Эффективная распределенная DAG (directed acyclic graph) модель вычислений (не только MapReduce)
- › "Ленивая" модель вычислений. Трансформации не выполняются до action. Spark строит весь граф и оптимизирует план: объединяет операции в этапы и избегает лишних проходов по данным.
- › Гибкие механизмы управления памятью:
  - › предпочтение хранения данных в памяти (это эффективнее с точки зрения доступа и производительности)
  - › возможность сброса данных на диск при нехватке памяти (это негативно сказывается на производительности, но зато позволяет не падать джобам, которые не помещаются в оперативную память)
- › возможность кэширования данных в памяти, на диске и комбинировано
- › различные форматы сериализации данных

# RDD

- › Базовая абстракция Spark
- › Resilient Distributed Dataset

# RDD

- › Базовая абстракция Spark
- › Resilient Distributed Dataset
- › **Dataset** - строго типизированный мешок с данными

# RDD

- › Базовая абстракция Spark
- › Resilient Distributed Dataset
- › **Dataset** - строго типизированный мешок с данными  
Более формально - неизменяемая (read-only) коллекция записей, разбитая на партии.

# RDD

- › Базовая абстракция Spark
- › Resilient Distributed Dataset
- › **Dataset** - строго типизированный мешок с данными  
Более формально - неизменяемая (read-only) коллекция записей, разбитая на партиции.
- › **Distributed** - распределенный по разным машинам. Датасет разбивается на партиции, которые могут храниться и обрабатываться по отдельности

# Действия над RDD

› Create: physical -> RDD

**Чтение из хранилища, создание из данных в памяти**

› Lazy Transformations: \*RDD -> RDD

**map, flatMap, filter, join, etc...**

› Action: RDD → результат в драйвер (collect, count) или запись в хранилище (saveAsTextFile)

**Запускает вычисление**

# RDD

- › Базовая абстракция Spark
- › Resilient Distributed Dataset
- › **Dataset** - строго типизированный мешок с данными  
Более формально - неизменяемая (read-only) коллекция записей, разбитая на партиции.
- › **Distributed** - распределенный по разным машинам. Датасет разбивается на партиции, которые могут храниться и обрабатываться по отдельности
- › **Resilient** - RDD является цепочкой преобразований партиций. Если какая-то партиция потерялась - её можно пересчитать. Для этого мы идем по графу вычислений и пересчитываем только то, что нам нужно.

# Spark SQL

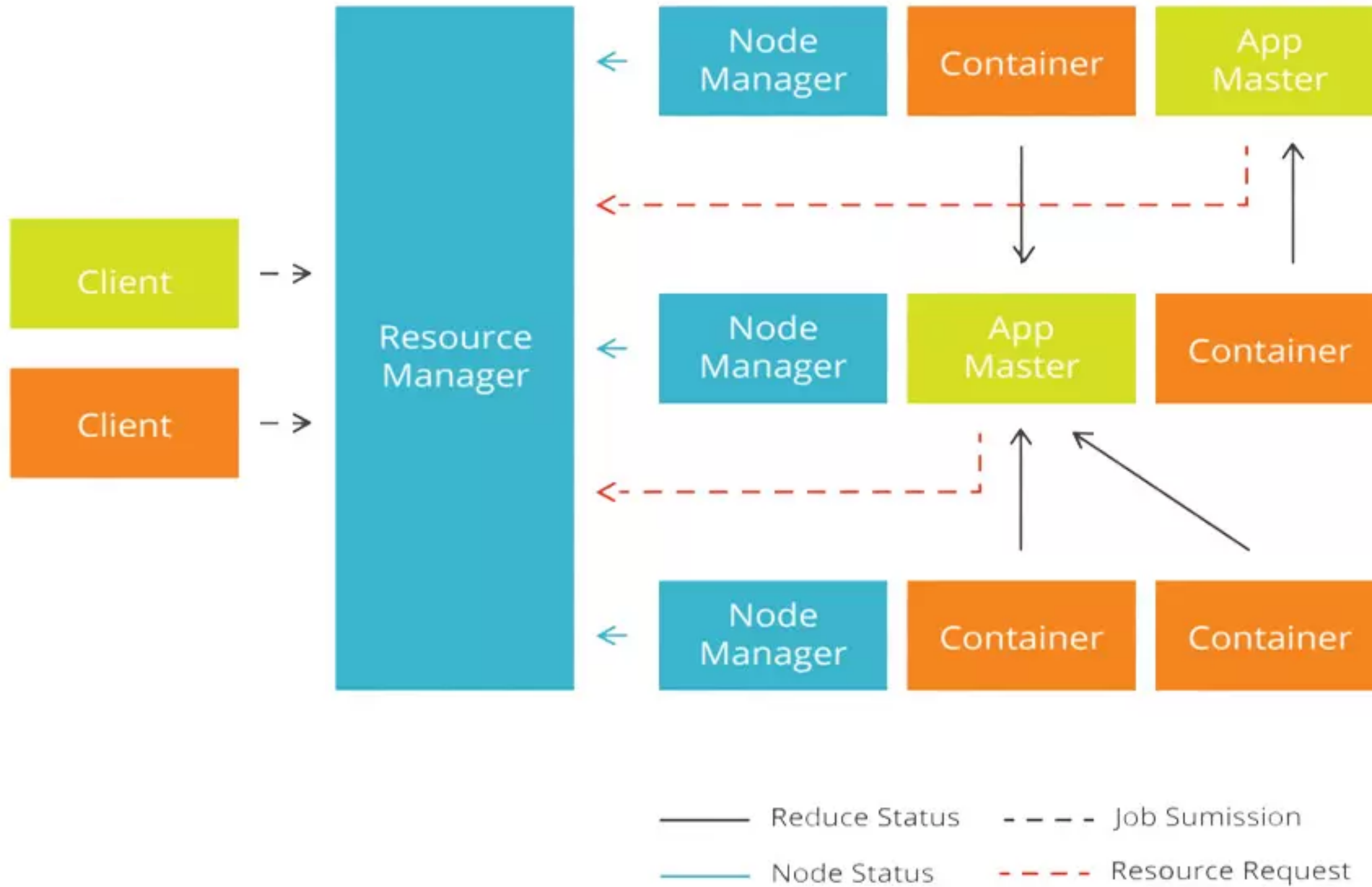
- › Spark SQL - модуль для структурированных данных: SQL + DataFrame API (+ Dataset в Scala/Java)
- › RDD под капотом
- › Есть более верхнеуровневый API, похожий на SQL
- › Оптимизатор запросов Catalyst
- › Работа с Dataframe и Dataset

**Демо 2.2**

# А кто за этим всем присматривать будет?

- › Есть кластер, в нем много машин с CPU и RAM
- › Есть много независимых процессов, которые надо запускать на кластере
- › Как решить, какой процесс запустить на какой машине?
- › Что делать, если свободных ресурсов меньше, чем требуется?
- › Как контролировать, что процесс занял не больше ресурсов, чем выделено?
- › Нужен отдельный сервис, который бы управлял ресурсами кластера

# Please welcome - Hadoop YARN



# Почему нельзя просто так взять, и выделить ресурсы?

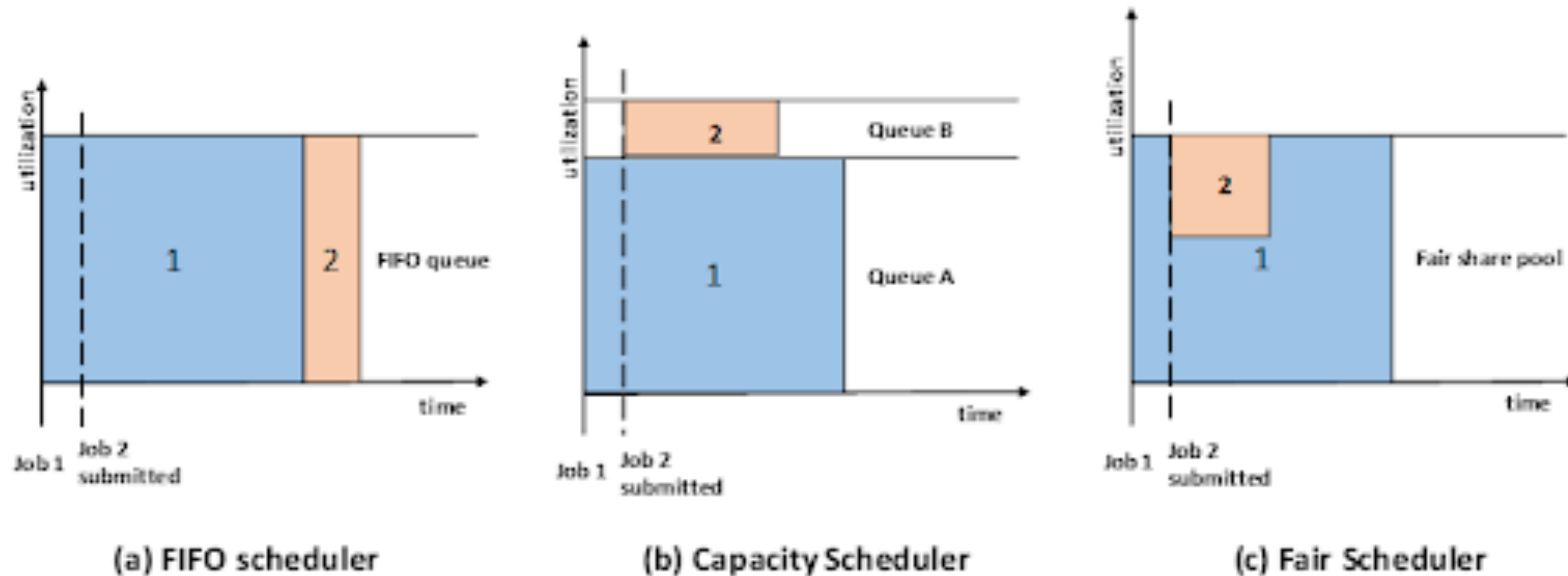
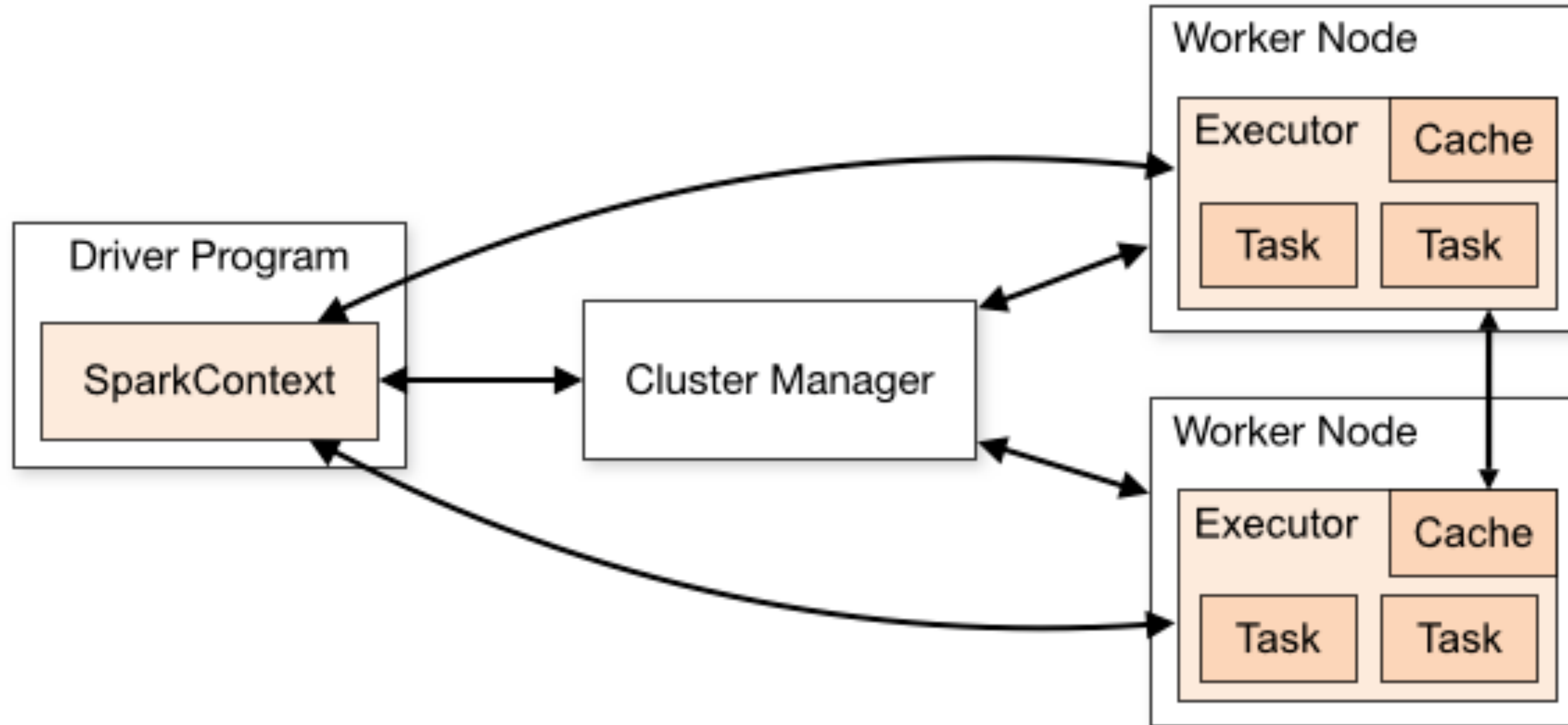


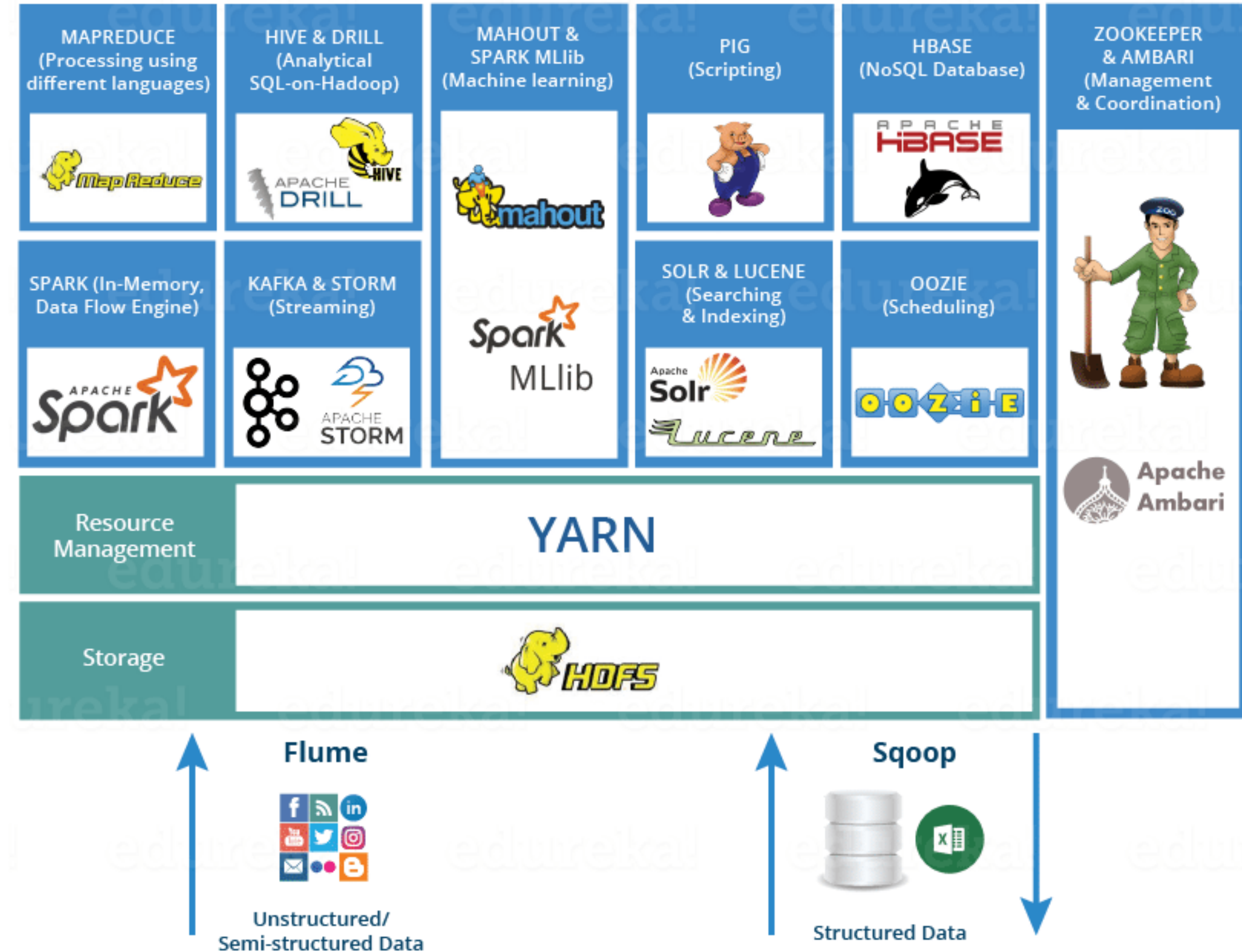
Figure 1: YARN Schedulers' cluster utilization vs. time

# Еще раз про Apache Spark



# Итого Hadoop-стэк

- › Hadoop Distributed File System (HDFS)
- › Hadoop MapReduce
- › Hadoop YARN
- + Apache Spark - удобный движок для расчетов
- + Hive - SQL движок для Big Data
- + HBase - wide-column NoSQL над HDFS
- + много самого разного



**That's all, Folks!**

